

Final Project Report

Project: Implement Client Script & UI Policy (Incident)
Document Reference: PRJ-INC-P6-002
Milestone: M6 - Documentation
Owner: Rahul (Team Lead)
Status: Submission Ready

1. Executive Summary

The project delivers client-side data-quality controls for the ServiceNow Incident form using one UI Policy (**High Impact Control**) and six client scripts. Short descriptions are validated on submit, Subcategory resets with Category, Critical priority is warned, High impact drives Urgency and ownership rules, and State cannot be edited inline from lists.

2. Objectives and Outcome

Objective	Delivered by	Status
Validate Short description on submit	onSubmit script	Implemented
Clear Subcategory on Category change	onChange script	Implemented
Warn for Priority 1 - Critical	onChange script	Implemented
High Impact UI Policy controls	UI Policy + 3 Actions	Implemented
Auto-set Urgency to High	onChange script	Implemented
Require Assigned To for High impact	UI Policy Action	Implemented
Block State list editing	onCellEdit script	Implemented
Allow State changes through form	Form unaffected	Implemented
Verify reverse-condition behaviour	Reverse if false + UAT-10	Test case defined
Documentation and verification	Repository package, test suite	Implemented

3. Deliverables Summary

Category	Items
Phase documents	17 Markdown documents in 6 phase folders
Implementation	6 client scripts, UI Policy JSON, setup script
Testing	12 UAT scenarios, 13 test cases, 15-check script simulation, repository suite
Packaging	README, FSD, report, DOCX + PDF for every document

4. Verification Summary

Check	Result
Repository structure and content checks	Run <code>python automated_tests/test_repository.py</code>
Offline client-script logic simulation	15 of 15 checks pass
Live ServiceNow UAT (12 scenarios)	To be executed by the team; record results in the UAT report

5. Business Impact

Expected qualitative benefits: improved data quality, fewer user-entry errors, better triage and routing consistency, controlled Incident updates and stronger reporting consistency. Quantitative benefits (for example reduction in re-opened or mis-categorised incidents) should be measured on the instance after go-live and are not claimed here.

6. Lessons Learned

1. Always guard `onChange` scripts with `isLoading`, otherwise saved data can be altered on open.
2. UI Policy Actions cannot set values, so pair them with a script when automation is required.
3. Client-side controls improve experience but do not replace server-side enforcement.
4. `onCellEdit` must call its callback exactly once or the list cell hangs.

7. Recommendations and Next Steps

1. Execute UAT and attach screenshots.
2. Add a Business Rule or Data Policy to enforce the same rules for imports and APIs.
3. Review choice values on the production instance before moving the update set.
4. Consider notifications for Critical incidents.

8. Team and Acknowledgements

Name	Role	Contribution
Rahul	Team Lead	Architecture, coordination, GitHub submission
Dinesh Kumar	Member	UI Policies and UI Policy Actions
Abisheknathan	Member	Client Scripts
Dakshanraj	Member	UAT testing, validation and final report